```
!pip install wandb timm --quiet
import wandb
wandb.login()
```

```
/usr/local/lib/python3.12/dist-packages/notebook/notebookapp.py:191: SyntaxWarning: invalid escape sequence '\/'
  | |_| | '_ \/ _` / _` |  _/ -_)
wandb: (1) Create a W&B account
wandb: (2) Use an existing W&B account
wandb: (3) Don't visualize my results
wandb: Enter your choice: 2
wandb: You chose 'Use an existing W&B account'
wandb: Logging into https://api.wandb.ai. (Learn how to deploy a W&B server locally: https://wandb.me/wandb-server)
wandb: Create a new API key at: https://wandb.ai/authorize?ref=models
wandb: Store your API key securely and do not share it.
wandb: Paste your API key and hit enter: ··********
wandb: No netrc file found, creating one.
wandb: Appending key for api.wandb.ai to your netrc file: /root/.netrc
wandb: Currently logged in as: mayankjain_25rco06 (mayankjain_25rco06-delhi-technological-university) to https://api.wandb.ai. Use
True
```

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import DataLoader, Subset
import torchvision
import torchvision.transforms as transforms
from torchvision.models import resnet18

import numpy as np
import matplotlib.pyplot as plt
import time
import copy
import wandb
from itertools import product as iter_product

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
print(f"Using device: {device}")

# Reproducibility
torch.manual_seed(42)
np.random.seed(42)
```

```
Using device: cuda
```

```
# --- Base transforms (no augmentation) ---
transform_base = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465),
                         (0.2470, 0.2435, 0.2616)),
])

# --- Augmented transforms (horizontal + vertical flip) ---
transform_augmented = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomVerticalFlip(p=0.5),
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465),
                         (0.2470, 0.2435, 0.2616)),
])

# --- Test/Val transform (no augmentation ever) ---
transform_test = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465),
                         (0.2470, 0.2435, 0.2616)),
])

# Download full CIFAR-10
full_train = torchvision.datasets.CIFAR10(root='./data', train=True,
                                          download=True, transform=None)
test_dataset_final = torchvision.datasets.CIFAR10(root='./data', train=False,
                                                  download=True, transform=transform_test)
```

```
# 80/10/10 split from the 50 000 training images
num_total = len(full_train)  # 50000
num_train = int(0.8 * num_total)  # 40000
num_val   = int(0.1 * num_total)  # 5000
num_test  = num_total - num_train - num_val  # 5000

indices = np.random.permutation(num_total)
train_idx = indices[:num_train]
val_idx   = indices[num_train:num_train + num_val]
test_idx  = indices[num_train + num_val:]


class TransformSubset(torch.utils.data.Dataset):
    """Wraps a Subset with a custom transform."""
    def __init__(self, dataset, indices, transform):
        self.dataset = dataset
        self.indices = indices
        self.transform = transform

    def __len__(self):
        return len(self.indices)

    def __getitem__(self, idx):
        img, label = self.dataset[self.indices[idx]]
        if self.transform:
            img = self.transform(img)
        return img, label


def get_dataloaders(augment=False, batch_size=128):
    """Return train, val, test DataLoaders."""
    train_tf = transform_augmented if augment else transform_base
    train_ds = TransformSubset(full_train, train_idx, train_tf)
    val_ds   = TransformSubset(full_train, val_idx,   transform_test)
    test_ds  = TransformSubset(full_train, test_idx,  transform_test)

    train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True,
                              num_workers=2, pin_memory=True)
    val_loader   = DataLoader(val_ds, batch_size=batch_size, shuffle=False,
                              num_workers=2, pin_memory=True)
```

```
    test_loader  = DataLoader(test_ds, batch_size=batch_size, shuffle=False,
                              num_workers=2, pin_memory=True)
    return train_loader, val_loader, test_loader


print(f"Train: {num_train} | Val: {num_val} | Test: {num_test}")
```

```
100%|██████████| 170M/170M [00:04<00:00, 40.9MB/s]
Train: 40000 | Val: 5000 | Test: 5000
```

```
class PatchEmbedding(nn.Module):
    """Split image into patches and project to embedding dimension."""
    def __init__(self, img_size=32, patch_size=4, in_channels=3, embed_dim=128):
        super().__init__()
        self.patch_size = patch_size
        self.num_patches = (img_size // patch_size) ** 2
        self.proj = nn.Conv2d(in_channels, embed_dim,
                              kernel_size=patch_size, stride=patch_size)

    def forward(self, x):
        # x: (B, C, H, W) -> (B, num_patches, embed_dim)
        x = self.proj(x)                      # (B, embed_dim, H', W')
        x = x.flatten(2).transpose(1, 2)      # (B, num_patches, embed_dim)
        return x


class MultiHeadSelfAttention(nn.Module):
    def __init__(self, embed_dim, num_heads, dropout=0.1):
        super().__init__()
        self.num_heads = num_heads
        self.head_dim = embed_dim // num_heads
        self.scale = self.head_dim ** -0.5

        self.qkv = nn.Linear(embed_dim, embed_dim * 3)
        self.proj = nn.Linear(embed_dim, embed_dim)
        self.attn_drop = nn.Dropout(dropout)
        self.proj_drop = nn.Dropout(dropout)

    def forward(self, x):
        B, N, C = x.shape
```

```
        qkv = self.qkv(x).reshape(B, N, 3, self.num_heads, self.head_dim)
        qkv = qkv.permute(2, 0, 3, 1, 4)     # (3, B, heads, N, head_dim)
        q, k, v = qkv[0], qkv[1], qkv[2]

        attn = (q @ k.transpose(-2, -1)) * self.scale
        attn = attn.softmax(dim=-1)
        attn = self.attn_drop(attn)

        x = (attn @ v).transpose(1, 2).reshape(B, N, C)
        x = self.proj(x)
        x = self.proj_drop(x)
        return x


class TransformerBlock(nn.Module):
    def __init__(self, embed_dim, num_heads, mlp_ratio=4.0, dropout=0.1):
        super().__init__()
        self.norm1 = nn.LayerNorm(embed_dim)
        self.attn = MultiHeadSelfAttention(embed_dim, num_heads, dropout)
        self.norm2 = nn.LayerNorm(embed_dim)
        self.mlp = nn.Sequential(
            nn.Linear(embed_dim, int(embed_dim * mlp_ratio)),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(int(embed_dim * mlp_ratio), embed_dim),
            nn.Dropout(dropout),
        )

    def forward(self, x):
        x = x + self.attn(self.norm1(x))   # Residual + attention
        x = x + self.mlp(self.norm2(x))    # Residual + FFN
        return x


class VisionTransformer(nn.Module):
    def __init__(self, img_size=32, patch_size=4, in_channels=3,
                 num_classes=10, embed_dim=128, depth=6,
                 num_heads=4, mlp_ratio=4.0, dropout=0.1):
        super().__init__()
        self.patch_embed = PatchEmbedding(img_size, patch_size,
                                          in_channels, embed_dim)
```

```
        num_patches = self.patch_embed.num_patches

        # Learnable CLS token
        self.cls_token = nn.Parameter(torch.zeros(1, 1, embed_dim))
        # Learnable positional encoding
        self.pos_embed = nn.Parameter(
            torch.zeros(1, num_patches + 1, embed_dim))
        self.pos_drop = nn.Dropout(dropout)

        # Transformer encoder blocks
        self.blocks = nn.Sequential(
            *[TransformerBlock(embed_dim, num_heads, mlp_ratio, dropout)
              for _ in range(depth)])

        self.norm = nn.LayerNorm(embed_dim)

        # Classification head
        self.head = nn.Sequential(
            nn.Linear(embed_dim, embed_dim),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(embed_dim, num_classes),
        )

        # Initialize
        nn.init.trunc_normal_(self.pos_embed, std=0.02)
        nn.init.trunc_normal_(self.cls_token, std=0.02)

    def forward(self, x):
        B = x.shape[0]
        x = self.patch_embed(x)                                # (B, N, D)
        cls_tokens = self.cls_token.expand(B, -1, -1)         # (B, 1, D)
        x = torch.cat([cls_tokens, x], dim=1)                 # (B, N+1, D)
        x = x + self.pos_embed                                # Add position
        x = self.pos_drop(x)
        x = self.blocks(x)                                    # Encoder
        x = self.norm(x)
        cls_out = x[:, 0]                                     # CLS token
        return self.head(cls_out)
```

```
def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)


# Quick test
vit_test = VisionTransformer().to(device)
print(f"ViT parameters: {count_parameters(vit_test):,}")
dummy = torch.randn(2, 3, 32, 32).to(device)
print(f"ViT output shape: {vit_test(dummy).shape}")
del vit_test, dummy
```

```
ViT parameters: 1,222,410
ViT output shape: torch.Size([2, 10])
```

```
def get_resnet18(num_classes=10):
    """ResNet-18 modified for 32×32 CIFAR images."""
    model = resnet18(weights=None)
    # Replace first conv: 7×7 stride 2 -> 3×3 stride 1 for small images
    model.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1, bias=False)
    model.maxpool = nn.Identity()  # Remove maxpool for 32×32
    model.fc = nn.Linear(model.fc.in_features, num_classes)
    return model

resnet_test = get_resnet18().to(device)
print(f"ResNet-18 parameters: {count_parameters(resnet_test):,}")
del resnet_test
```

```
ResNet-18 parameters: 11,173,962
```

```
class FocalLoss(nn.Module):
    """Focal Loss for handling class imbalance / hard examples."""
    def __init__(self, alpha=1.0, gamma=2.0, reduction='mean'):
        super().__init__()
        self.alpha = alpha
        self.gamma = gamma
        self.reduction = reduction

    def forward(self, inputs, targets):
        ce_loss = F.cross_entropy(inputs, targets, reduction='none')
```

```
        pt = torch.exp(-ce_loss)
        focal_loss = self.alpha * (1 - pt) ** self.gamma * ce_loss
        if self.reduction == 'mean':
            return focal_loss.mean()
        return focal_loss.sum()


def get_loss_fn(loss_name):
    """Return loss function by name."""
    if loss_name == 'cross_entropy':
        return nn.CrossEntropyLoss()
    elif loss_name == 'label_smoothing':
        return nn.CrossEntropyLoss(label_smoothing=0.1)
    elif loss_name == 'focal':
        return FocalLoss(alpha=1.0, gamma=2.0)
    else:
        raise ValueError(f"Unknown loss: {loss_name}")


def get_optimizer(model, opt_name, lr):
    """Return optimizer by name."""
    if opt_name == 'sgd':
        return optim.SGD(model.parameters(), lr=lr, momentum=0.9,
                         weight_decay=1e-4)
    elif opt_name == 'rmsprop':
        return optim.RMSprop(model.parameters(), lr=lr, weight_decay=1e-4)
    elif opt_name == 'adam':
        return optim.Adam(model.parameters(), lr=lr, weight_decay=1e-4)
    else:
        raise ValueError(f"Unknown optimizer: {opt_name}")
```

```
def train_one_epoch(model, loader, criterion, optimizer):
    model.train()
    running_loss, correct, total = 0.0, 0, 0

    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
```

```
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)
        _, preds = outputs.max(1)
        correct += preds.eq(labels).sum().item()
        total += labels.size(0)

    return running_loss / total, 100.0 * correct / total


@torch.no_grad()
def evaluate(model, loader, criterion):
    model.eval()
    running_loss, correct, total = 0.0, 0, 0

    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        loss = criterion(outputs, labels)

        running_loss += loss.item() * images.size(0)
        _, preds = outputs.max(1)
        correct += preds.eq(labels).sum().item()
        total += labels.size(0)

    return running_loss / total, 100.0 * correct / total


def train_model(model, train_loader, val_loader, criterion, optimizer,
                scheduler, num_epochs, run_name):
    """Full training loop with W&B logging."""
    best_val_acc = 0.0
    best_model_wts = copy.deepcopy(model.state_dict())
    history = {'train_loss': [], 'val_loss': [],
               'train_acc': [], 'val_acc': []}

    start_time = time.time()

    for epoch in range(num_epochs):
```

```
        train_loss, train_acc = train_one_epoch(
            model, train_loader, criterion, optimizer)
        val_loss, val_acc = evaluate(model, val_loader, criterion)

        if scheduler is not None:
            scheduler.step()

        # Log to W&B
        wandb.log({
            "epoch": epoch + 1,
            "train_loss": train_loss,
            "train_acc": train_acc,
            "val_loss": val_loss,
            "val_acc": val_acc,
            "lr": optimizer.param_groups[0]['lr'],
        })

        history['train_loss'].append(train_loss)
        history['val_loss'].append(val_loss)
        history['train_acc'].append(train_acc)
        history['val_acc'].append(val_acc)

        if val_acc > best_val_acc:
            best_val_acc = val_acc
            best_model_wts = copy.deepcopy(model.state_dict())

        if (epoch + 1) % 5 == 0 or epoch == 0:
            print(f"  Epoch {epoch+1:3d}/{num_epochs} | "
                  f"Train Loss: {train_loss:.4f} Acc: {train_acc:.2f}% | "
                  f"Val Loss: {val_loss:.4f} Acc: {val_acc:.2f}%")

    total_time = time.time() - start_time
    model.load_state_dict(best_model_wts)

    print(f"  Training time: {total_time:.1f}s | Best Val Acc: {best_val_acc:.2f}%")
    wandb.log({"best_val_acc": best_val_acc, "training_time_s": total_time})

    return model, history, total_time, best_val_acc
```

```
# This runs all 36 configurations (2 models × 2 aug × 3 loss × 3 opt).
```

```
# Estimated time on Colab GPU: ~3–5 hours.
# To do a quick test first, set QUICK_TEST = True below.

QUICK_TEST = False   # Set True to run only 1 config per model for sanity check

NUM_EPOCHS      = 20
BATCH_SIZE      = 128
LR_VIT          = 1e-3
LR_RESNET       = 0.01

MODEL_NAMES     = ['vit', 'resnet18']
AUG_OPTIONS     = [False, True]
LOSS_NAMES      = ['cross_entropy', 'label_smoothing', 'focal']
OPT_NAMES       = ['sgd', 'rmsprop', 'adam']

if QUICK_TEST:
    MODEL_NAMES = ['vit', 'resnet18']
    AUG_OPTIONS = [False]
    LOSS_NAMES  = ['cross_entropy']
    OPT_NAMES   = ['adam']
    NUM_EPOCHS  = 5

# Store all results
all_results = []

WANDB_PROJECT = "experiment10-vit-resnet"  # Change if you like

configs = list(iter_product(MODEL_NAMES, AUG_OPTIONS, LOSS_NAMES, OPT_NAMES))
print(f"Total configurations to run: {len(configs)}")

for i, (model_name, augment, loss_name, opt_name) in enumerate(configs):
    aug_tag = "aug" if augment else "noaug"
    run_name = f"{model_name}_{aug_tag}_{loss_name}_{opt_name}"
    print(f"\n{'='*60}")
    print(f"[{i+1}/{len(configs)}] {run_name}")
    print(f"{'='*60}")

    # Initialize W&B run
    wandb.init(
        project=WANDB_PROJECT,
        name=run_name,
```

```
    name=run_name,
    config={
        "model": model_name,
        "augmentation": augment,
        "loss_function": loss_name,
        "optimizer": opt_name,
        "epochs": NUM_EPOCHS,
        "batch_size": BATCH_SIZE,
        "lr": LR_VIT if model_name == 'vit' else LR_RESNET,
    },
    reinit=True,
)

# Data
train_loader, val_loader, test_loader = get_dataloaders(
    augment=augment, batch_size=BATCH_SIZE)

# Model
if model_name == 'vit':
    model = VisionTransformer(
        img_size=32, patch_size=4, in_channels=3, num_classes=10,
        embed_dim=128, depth=6, num_heads=4, mlp_ratio=4.0, dropout=0.1
    ).to(device)
    lr = LR_VIT
else:
    model = get_resnet18(num_classes=10).to(device)
    lr = LR_RESNET

# Loss & optimizer
criterion = get_loss_fn(loss_name)
optimizer = get_optimizer(model, opt_name, lr)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=NUM_EPOCHS)

# Train
model, history, train_time, best_val = train_model(
    model, train_loader, val_loader, criterion, optimizer,
    scheduler, NUM_EPOCHS, run_name)

# Test
test_loss, test_acc = evaluate(model, test_loader, criterion)
print(f"  >> Test Accuracy: {test_acc:.2f}%")
```

```
        wandb.log({"test_acc": test_acc, "test_loss": test_loss})

        # Save model checkpoint
        ckpt_path = f"checkpoints/{run_name}.pt"
        import os; os.makedirs("checkpoints", exist_ok=True)
        torch.save(model.state_dict(), ckpt_path)

        all_results.append({
            "run_name": run_name,
            "model": model_name,
            "augmentation": augment,
            "loss": loss_name,
            "optimizer": opt_name,
            "best_val_acc": best_val,
            "test_acc": test_acc,
            "train_time_s": train_time,
            "params": count_parameters(model),
        })

        wandb.finish()

    print("\n\nAll experiments completed!")
```

```
Total configurations to run: 36

============================================================
[1/36] vit_noaug_cross_entropy_sgd
============================================================
wandb: WARNING Using a boolean value for 'reinit' is deprecated. Use 'return_previous' or 'finish_previous' instead.
Tracking run with wandb version 0.25.1
Run data is saved locally in /content/wandb/run-20260416_121345-s49e3diq
Syncing run vit_noaug_cross_entropy_sgd to Weights & Biases (docs)
View project at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet
View run at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet/runs/s49e3diq
  Epoch   1/20 | Train Loss: 2.0517 Acc: 24.08% | Val Loss: 1.8809 Acc: 30.00%
  Epoch   5/20 | Train Loss: 1.6541 Acc: 38.65% | Val Loss: 1.5696 Acc: 42.90%
  Epoch  10/20 | Train Loss: 1.4778 Acc: 46.25% | Val Loss: 1.4082 Acc: 49.00%
  Epoch  15/20 | Train Loss: 1.3845 Acc: 49.57% | Val Loss: 1.3696 Acc: 50.42%
  Epoch  20/20 | Train Loss: 1.3481 Acc: 50.95% | Val Loss: 1.3318 Acc: 52.08%
  Training time: 375.7s | Best Val Acc: 52.32%
  >> Test Accuracy: 51.06%
```

Run history:

best_val_acc	_
epoch	▁▁▁▂▂▂▂▂▃▃▃▃▄▄▄▄▅▅▅▅▆▆▆▆▇▇▇▇▇▇▇▇▇▇▇▇
lr	▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇
test_acc	_
test_loss	_
train_acc	▁▃▄▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅
train_loss	▇▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅
training_time_s	_
val_acc	▁▃▄▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅
val_loss	▇▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅▅

Run summary:

best_val_acc	52.32
epoch	20
lr	0

test_acc	51.06
test_loss	1.34939
train_acc	50.9475
train_loss	1.34808
training_time_s	375.72302
val_acc	52.08
val_loss	1.33178

View run **vit_noaug_cross_entropy_sgd** at: https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet/runs/s49e3diq
View project at: https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet
Synced 5 W&B file(s), 0 media file(s), 0 artifact file(s) and 0 other file(s)
Find logs at: `./wandb/run-20260416_121345-s49e3diq/logs`

```
============================================================
[2/36] vit_noaug_cross_entropy_rmsprop
============================================================
```

Tracking run with wandb version 0.25.1
Run data is saved locally in `/content/wandb/run-20260416_122007-ow1oif6w`
Syncing run **vit_noaug_cross_entropy_rmsprop** to Weights & Biases (docs)
View project at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet
View run at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet/runs/ow1oif6w

```
  Epoch   1/20 | Train Loss: 1.9628 Acc: 26.30% | Val Loss: 1.7591 Acc: 33.66%
  Epoch   5/20 | Train Loss: 1.3796 Acc: 49.89% | Val Loss: 1.3653 Acc: 51.14%
  Epoch  10/20 | Train Loss: 1.1486 Acc: 58.58% | Val Loss: 1.2242 Acc: 57.14%
  Epoch  15/20 | Train Loss: 0.9846 Acc: 64.46% | Val Loss: 1.0766 Acc: 61.44%
  Epoch  20/20 | Train Loss: 0.9087 Acc: 67.36% | Val Loss: 1.0478 Acc: 62.70%
  Training time: 376.5s | Best Val Acc: 62.70%
  >> Test Accuracy: 62.04%
```

Run history:

best_val_acc	_
epoch	
lr	
test_acc	_
test_loss	_
train_acc	
train_loss	
training_time_s	_

val_acc
val_loss

Run summary:

best_val_acc	62.7
epoch	20
lr	0
test_acc	62.04
test_loss	1.06746
train_acc	67.36
train_loss	0.90867
training_time_s	376.47005
val_acc	62.7
val_loss	1.04779

View run **vit_noaug_cross_entropy_rmsprop** at: https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet/runs/ow1oif6w
View project at: https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet
Synced 5 W&B file(s), 0 media file(s), 0 artifact file(s) and 0 other file(s)
Find logs at: `./wandb/run-20260416_122007-ow1oif6w/logs`

```
============================================================
[3/36] vit_noaug_cross_entropy_adam
============================================================
```
Tracking run with wandb version 0.25.1
Run data is saved locally in `/content/wandb/run-20260416_122628-t49iudze`
Syncing run **vit_noaug_cross_entropy_adam** to Weights & Biases (docs)
View project at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet
View run at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet/runs/t49iudze

```
  Epoch   1/20 | Train Loss: 1.8099 Acc: 31.95% | Val Loss: 1.6042 Acc: 39.08%
  Epoch   5/20 | Train Loss: 1.2449 Acc: 55.00% | Val Loss: 1.2331 Acc: 55.02%
  Epoch  10/20 | Train Loss: 1.0062 Acc: 63.79% | Val Loss: 1.0739 Acc: 61.90%
  Epoch  15/20 | Train Loss: 0.8057 Acc: 71.01% | Val Loss: 0.9727 Acc: 65.52%
  Epoch  20/20 | Train Loss: 0.7014 Acc: 74.91% | Val Loss: 0.9637 Acc: 66.46%
  Training time: 376.9s | Best Val Acc: 66.80%
  >> Test Accuracy: 67.50%
```

Run history:

best_val_acc	_
epoch	▁▁▁▁▁▁▁▁▁▁▁▁▁▁▁▁▁▁▁▁
lr	▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇
test_acc	_
test_loss	_
train_acc	▁▁▁▁▁▁▁▁▁▁▁▁▁▁▁▁▁▁▁▁
train_loss	▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇
training_time_s	_
val_acc	▁▁▁▁▁▁▁▁▁▁▁▁▁▁▁▁▁▁▁▁
val_loss	▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇

Run summary:

best_val_acc	66.8
epoch	20
lr	0
test_acc	67.5
test_loss	0.94327
train_acc	74.91
train_loss	0.70138
training_time_s	376.90236
val_acc	66.46
val_loss	0.96374

View run **vit_noaug_cross_entropy_adam** at: https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet/runs/t49iudze
View project at: https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet
Synced 5 W&B file(s), 0 media file(s), 0 artifact file(s) and 0 other file(s)
Find logs at: `./wandb/run-20260416_122628-t49iudze/logs`

```
============================================================
[4/36] vit_noaug_label_smoothing_sgd
============================================================
```

Tracking run with wandb version 0.25.1
Run data is saved locally in `/content/wandb/run-20260416_123252-nicenkc5`
Syncing run **vit_noaug_label_smoothing_sgd** to Weights & Biases (docs)
View project at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet
View run at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet/runs/nicenkc5

```
Epoch   1/20 | Train Loss: 2.1115 Acc: 23.65% | Val Loss: 2.0004 Acc: 27.86%
Epoch   5/20 | Train Loss: 1.8252 Acc: 37.07% | Val Loss: 1.7763 Acc: 39.20%
Epoch  10/20 | Train Loss: 1.6862 Acc: 44.60% | Val Loss: 1.6511 Acc: 46.88%
Epoch  15/20 | Train Loss: 1.6170 Acc: 47.95% | Val Loss: 1.5944 Acc: 49.10%
Epoch  20/20 | Train Loss: 1.5921 Acc: 49.00% | Val Loss: 1.5726 Acc: 49.94%
Training time: 375.9s | Best Val Acc: 49.94%
>> Test Accuracy: 49.82%
```

Run history:

best_val_acc	_
epoch	_
lr	_
test_acc	_
test_loss	_
train_acc	_
train_loss	_
training_time_s	_
val_acc	_
val_loss	_

Run summary:

best_val_acc	49.94
epoch	20
lr	0
test_acc	49.82
test_loss	1.59123
train_acc	48.9975
train_loss	1.59205
training_time_s	375.89635
val_acc	49.94
val_loss	1.57258

View run **vit_noaug_label_smoothing_sgd** at: https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet/runs/nicenkc5
View project at: https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet

```
Synced 5 W&B file(s), 0 media file(s), 0 artifact file(s) and 0 other file(s)
Find logs at: ./wandb/run-20260416_123252-nicenkc5/logs

============================================================
[5/36] vit_noaug_label_smoothing_rmsprop
============================================================
Tracking run with wandb version 0.25.1
Run data is saved locally in /content/wandb/run-20260416_123912-4miintqn
Syncing run vit_noaug_label_smoothing_rmsprop to Weights & Biases (docs)
View project at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet
View run at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet/runs/4miintqn
  Epoch   1/20 | Train Loss: 2.0572 Acc: 24.52% | Val Loss: 1.9187 Acc: 33.08%
  Epoch   5/20 | Train Loss: 1.5941 Acc: 49.06% | Val Loss: 1.6100 Acc: 48.56%
  Epoch  10/20 | Train Loss: 1.4171 Acc: 57.84% | Val Loss: 1.4355 Acc: 55.76%
  Epoch  15/20 | Train Loss: 1.3065 Acc: 63.27% | Val Loss: 1.3697 Acc: 59.32%
  Epoch  20/20 | Train Loss: 1.2557 Acc: 65.83% | Val Loss: 1.3333 Acc: 61.58%
  Training time: 374.5s | Best Val Acc: 61.58%
  >> Test Accuracy: 62.16%
```

Run history:

best_val_acc	▁
epoch	▁▁▂▂▂▂▂▃▃▃▃▄▄▄▄▅▅▅▅▆▆▆▆▇▇▇

test_loss	1.33358
train_acc	65.83
train_loss	1.25566
training_time_s	374.47606
val_acc	61.58
val_loss	1.33326

View run **vit_noaug_label_smoothing_rmsprop** at: https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet/runs/4miintqn
View project at: https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet
Synced 5 W&B file(s), 0 media file(s), 0 artifact file(s) and 0 other file(s)
Find logs at: `./wandb/run-20260416_123912-4miintqn/logs`

```
============================================================
[6/36] vit_noaug_label_smoothing_adam
============================================================
```

Tracking run with wandb version 0.25.1
Run data is saved locally in `/content/wandb/run-20260416_124530-akgwiy0k`
Syncing run **vit_noaug_label_smoothing_adam** to Weights & Biases (docs)
View project at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet
View run at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet/runs/akgwiy0k

```
  Epoch   1/20 | Train Loss: 1.9115 Acc: 32.06% | Val Loss: 1.7668 Acc: 39.52%
  Epoch   5/20 | Train Loss: 1.4708 Acc: 55.15% | Val Loss: 1.4309 Acc: 56.08%
  Epoch  10/20 | Train Loss: 1.2820 Acc: 64.31% | Val Loss: 1.2911 Acc: 63.64%
  Epoch  15/20 | Train Loss: 1.1327 Acc: 71.78% | Val Loss: 1.2284 Acc: 67.64%
  Epoch  20/20 | Train Loss: 1.0574 Acc: 75.31% | Val Loss: 1.2139 Acc: 67.86%
  Training time: 378.3s | Best Val Acc: 68.12%
  >> Test Accuracy: 67.42%
```

Run history:

best_val_acc	_
epoch	▁▁▂▂▂▂▃▃▃▃▄▄▄▄▅▅▅▅▆▆▆▆▇▇▇

val_acc
val_loss

Run summary:

best_val_acc	68.12
epoch	20
lr	0
test_acc	67.42
test_loss	1.22189
train_acc	75.3125
train_loss	1.05742
training_time_s	378.32431
val_acc	67.86
val_loss	1.21393

View run **vit_noaug_label_smoothing_adam** at: https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet/runs/akgwiy0k
View project at: https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet
Synced 5 W&B file(s), 0 media file(s), 0 artifact file(s) and 0 other file(s)
Find logs at: `./wandb/run-20260416_124530-akgwiy0k/logs`

```
============================================================
[7/36] vit_noaug_focal_sgd
============================================================
```

Tracking run with wandb version 0.25.1
Run data is saved locally in `/content/wandb/run-20260416_125154-wecdesgr`
Syncing run **vit_noaug_focal_sgd** to Weights & Biases (docs)
View project at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet
View run at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet/runs/wecdesgr

```
  Epoch   1/20 | Train Loss: 1.5410 Acc: 24.94% | Val Loss: 1.3616 Acc: 30.56%
  Epoch   5/20 | Train Loss: 1.1407 Acc: 39.06% | Val Loss: 1.0717 Acc: 43.10%
  Epoch  10/20 | Train Loss: 0.9992 Acc: 44.96% | Val Loss: 0.9397 Acc: 48.00%
  Epoch  15/20 | Train Loss: 0.9201 Acc: 48.60% | Val Loss: 0.9041 Acc: 50.08%
  Epoch  20/20 | Train Loss: 0.8934 Acc: 49.74% | Val Loss: 0.8937 Acc: 50.46%
  Training time: 375.2s | Best Val Acc: 50.46%
  >> Test Accuracy: 50.28%
```

Run history:

best_val_acc	_
epoch	▁▁▂▂▂▂▂▃▃▃▃▃▄▄▄▄▅▅▅

```
Epoch   5/20 | Train Loss: 0.9238 Acc: 47.94% | Val Loss: 0.8854 Acc: 49.32%
Epoch  10/20 | Train Loss: 0.7315 Acc: 56.28% | Val Loss: 0.7552 Acc: 55.82%
Epoch  15/20 | Train Loss: 0.6058 Acc: 61.90% | Val Loss: 0.6886 Acc: 59.22%
Epoch  20/20 | Train Loss: 0.5525 Acc: 64.03% | Val Loss: 0.6484 Acc: 61.24%
Training time: 377.7s | Best Val Acc: 61.40%
>> Test Accuracy: 61.22%
```

Run history:

best_val_acc	_
epoch	▁▂▃▄▅▆▇▇▇

Find logs at: `./wandb/run-20260416_125814-7y1mhej2/logs`

```
============================================================
[9/36] vit_noaug_focal_adam
============================================================
```

Tracking run with wandb version 0.25.1
Run data is saved locally in `/content/wandb/run-20260416_130546-rxilpxip`
Syncing run **vit_noaug_focal_adam** to Weights & Biases (docs)
View project at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet
View run at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet/runs/rxilpxip

```
  Epoch   1/20 | Train Loss: 1.3065 Acc: 31.66% | Val Loss: 1.1332 Acc: 37.10%
  Epoch   5/20 | Train Loss: 0.7951 Acc: 53.42% | Val Loss: 0.7624 Acc: 54.74%
  Epoch  10/20 | Train Loss: 0.5936 Acc: 62.81% | Val Loss: 0.6401 Acc: 62.48%
  Epoch  15/20 | Train Loss: 0.4481 Acc: 69.55% | Val Loss: 0.5608 Acc: 66.16%
  Epoch  20/20 | Train Loss: 0.3720 Acc: 73.23% | Val Loss: 0.5461 Acc: 67.98%
  Training time: 375.2s | Best Val Acc: 67.98%
  >> Test Accuracy: 67.86%
```

Run history:

best_val_acc	_
epoch	▁▁▁▂▂▂▂▂▂▂▂▂▂▂▂▂▂▂▂▂
lr	▇▇▇

test_loss	0.53868
train_acc	73.23
train_loss	0.37203
training_time_s	375.21915
val_acc	67.98
val_loss	0.54611

View run **vit_noaug_focal_adam** at: https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet/runs/rxilpxip
View project at: https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet
Synced 5 W&B file(s), 0 media file(s), 0 artifact file(s) and 0 other file(s)
Find logs at: `./wandb/run-20260416_130546-rxilpxip/logs`

```
============================================================
[10/36] vit_aug_cross_entropy_sgd
============================================================
```

Tracking run with wandb version 0.25.1
Run data is saved locally in `/content/wandb/run-20260416_131256-3mbgidmw`
Syncing run **vit_aug_cross_entropy_sgd** to Weights & Biases (docs)
View project at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet
View run at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet/runs/3mbgidmw

```
  Epoch   1/20 | Train Loss: 2.0571 Acc: 23.14% | Val Loss: 1.9393 Acc: 28.26%
  Epoch   5/20 | Train Loss: 1.7248 Acc: 35.70% | Val Loss: 1.7020 Acc: 37.34%
  Epoch  10/20 | Train Loss: 1.5783 Acc: 41.96% | Val Loss: 1.5056 Acc: 45.48%
  Epoch  15/20 | Train Loss: 1.4868 Acc: 45.81% | Val Loss: 1.4458 Acc: 47.50%
  Epoch  20/20 | Train Loss: 1.4569 Acc: 46.70% | Val Loss: 1.4250 Acc: 48.36%
  Training time: 404.6s | Best Val Acc: 48.36%
  >> Test Accuracy: 48.00%
```

Run history:

best_val_acc	▁
epoch	▁▁▂▂▂▃▃▃▄▄▄▅▅▅▆▆▆▇▇▇

val_loss

Run summary:

best_val_acc	48.36
epoch	20
lr	0
test_acc	48
test_loss	1.4528
train_acc	46.6975
train_loss	1.4569
training_time_s	404.60446
val_acc	48.36
val_loss	1.42495

View run **vit_aug_cross_entropy_sgd** at: https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet/runs/3mbgidmw
View project at: https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet
Synced 5 W&B file(s), 0 media file(s), 0 artifact file(s) and 0 other file(s)
Find logs at: `./wandb/run-20260416_131256-3mbgidmw/logs`

```
============================================================
[11/36] vit_aug_cross_entropy_rmsprop
============================================================
```

Tracking run with wandb version 0.25.1
Run data is saved locally in `/content/wandb/run-20260416_131948-cxxanhtf`
Syncing run **vit_aug_cross_entropy_rmsprop** to Weights & Biases (docs)
View project at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet
View run at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet/runs/cxxanhtf

```
  Epoch   1/20 | Train Loss: 1.9872 Acc: 25.36% | Val Loss: 1.9547 Acc: 25.88%
  Epoch   5/20 | Train Loss: 1.4721 Acc: 46.06% | Val Loss: 1.4743 Acc: 45.32%
  Epoch  10/20 | Train Loss: 1.2637 Acc: 54.10% | Val Loss: 1.2769 Acc: 53.96%
  Epoch  15/20 | Train Loss: 1.1340 Acc: 58.82% | Val Loss: 1.1600 Acc: 58.64%
  Epoch  20/20 | Train Loss: 1.0743 Acc: 60.87% | Val Loss: 1.1226 Acc: 60.46%
  Training time: 404.1s | Best Val Acc: 60.46%
  >> Test Accuracy: 59.08%
```

Run history:

best_val_acc	_
epoch	▁▁▂▂▂▂▂▂▃▃▃▃▃▃▄▄▄▄▅▅▅

```
Epoch   5/20 | Train Loss: 1.3406 Acc: 51.13% | Val Loss: 1.3112 Acc: 52.32%
Epoch  10/20 | Train Loss: 1.1099 Acc: 59.98% | Val Loss: 1.0961 Acc: 60.44%
Epoch  15/20 | Train Loss: 0.9333 Acc: 66.55% | Val Loss: 0.9958 Acc: 64.86%
Epoch  20/20 | Train Loss: 0.8444 Acc: 69.67% | Val Loss: 0.9541 Acc: 66.32%
Training time: 402.2s | Best Val Acc: 66.56%
>> Test Accuracy: 66.90%
```

Run history:

best_val_acc	_
epoch	▁▂▃▄▅▆▇▇▇

```
Find logs at: ./wandb/run-20260416_132636-d00xfbop/logs

============================================================
[13/36] vit_aug_label_smoothing_sgd
============================================================
Tracking run with wandb version 0.25.1
Run data is saved locally in /content/wandb/run-20260416_133355-vc6d4n2q
Syncing run vit_aug_label_smoothing_sgd to Weights & Biases (docs)
View project at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet
View run at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet/runs/vc6d4n2q
  Epoch   1/20 | Train Loss: 2.1136 Acc: 23.47% | Val Loss: 2.0250 Acc: 28.08%
  Epoch   5/20 | Train Loss: 1.8564 Acc: 35.40% | Val Loss: 1.8148 Acc: 37.36%
  Epoch  10/20 | Train Loss: 1.7499 Acc: 40.92% | Val Loss: 1.7529 Acc: 41.06%
  Epoch  15/20 | Train Loss: 1.6847 Acc: 44.32% | Val Loss: 1.6740 Acc: 46.02%
  Epoch  20/20 | Train Loss: 1.6592 Acc: 45.60% | Val Loss: 1.6420 Acc: 47.38%
  Training time: 394.0s | Best Val Acc: 47.44%
  >> Test Accuracy: 45.18%
```

Run history:

best_val_acc	_
epoch	▁▁▂▂▃▃▄▄▅▅▆▆▇▇▇

test_loss	1.66904
train_acc	45.605
train_loss	1.65915
training_time_s	393.98909
val_acc	47.38
val_loss	1.64203

View run **vit_aug_label_smoothing_sgd** at: https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet/runs/vc6d4n2q
View project at: https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet
Synced 5 W&B file(s), 0 media file(s), 0 artifact file(s) and 0 other file(s)
Find logs at: `./wandb/run-20260416_133355-vc6d4n2q/logs`

```
============================================================
[14/36] vit_aug_label_smoothing_rmsprop
============================================================
```

Tracking run with wandb version 0.25.1
Run data is saved locally in `/content/wandb/run-20260416_134034-f5dg4buz`
Syncing run **vit_aug_label_smoothing_rmsprop** to Weights & Biases (docs)
View project at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet
View run at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet/runs/f5dg4buz

```
  Epoch   1/20 | Train Loss: 2.0444 Acc: 25.83% | Val Loss: 1.9733 Acc: 28.58%
  Epoch   5/20 | Train Loss: 1.6583 Acc: 45.56% | Val Loss: 1.6464 Acc: 45.58%
  Epoch  10/20 | Train Loss: 1.4989 Acc: 53.63% | Val Loss: 1.5028 Acc: 52.70%
  Epoch  15/20 | Train Loss: 1.3986 Acc: 58.46% | Val Loss: 1.4166 Acc: 57.62%
  Epoch  20/20 | Train Loss: 1.3513 Acc: 60.88% | Val Loss: 1.3783 Acc: 59.26%
  Training time: 397.0s | Best Val Acc: 59.32%
  >> Test Accuracy: 59.22%
```

Run history:

best_val_acc	▁
epoch	▁▁▂▂▂▂▂▂▃▃▃▃▃▃▄▄▄▄▄▄▅▅▅▅▅▅▆▆▆▆▆▆▇▇▇

val_loss

Run summary:

best_val_acc	59.32
epoch	20
lr	0
test_acc	59.22
test_loss	1.38048
train_acc	60.875
train_loss	1.35127
training_time_s	396.97684
val_acc	59.26
val_loss	1.37828

View run **vit_aug_label_smoothing_rmsprop** at: https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet/runs/f5dg4buz
View project at: https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet
Synced 5 W&B file(s), 0 media file(s), 0 artifact file(s) and 0 other file(s)
Find logs at: `./wandb/run-20260416_134034-f5dg4buz/logs`

```
============================================================
[15/36] vit_aug_label_smoothing_adam
============================================================
```

Tracking run with wandb version 0.25.1
Run data is saved locally in `/content/wandb/run-20260416_134716-grrg0n6h`
Syncing run **vit_aug_label_smoothing_adam** to Weights & Biases (docs)
View project at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet
View run at https://wandb.ai/mayankjain_25rco06-delhi-technological-university/experiment10-vit-resnet/runs/grrg0n6h

```
  Epoch   1/20 | Train Loss: 1.9319 Acc: 31.21% | Val Loss: 1.8764 Acc: 33.66%
  Epoch   5/20 | Train Loss: 1.5418 Acc: 51.84% | Val Loss: 1.5176 Acc: 53.22%
  Epoch  10/20 | Train Loss: 1.3808 Acc: 59.74% | Val Loss: 1.3722 Acc: 60.72%
  Epoch  15/20 | Train Loss: 1.2555 Acc: 65.78% | Val Loss: 1.2991 Acc: 63.92%
  Epoch  20/20 | Train Loss: 1.1956 Acc: 68.67% | Val Loss: 1.2687 Acc: 65.74%
  Training time: 398.5s | Best Val Acc: 65.74%
  >> Test Accuracy: 65.30%
```

Run history:

best_val_acc
epoch
lr
test_acc
test_loss
train_acc
train_loss
training_time_s
val_acc
val_loss